



南開大學
Nankai University

MettEagle: Costs and Benefits of Implementing Containers on Microkernels

于昊汐 陈安琪 孙俪菲



目录

CONTENTS

01

背景&问题动机

02

Linux容器的本质问题

03

为什么是内核?

04

MettEagle的整体架构

05

隔离机制对比

06

安全性评估

07

性能评估

08

总结

允公允能 日新月异



南開大學
Nankai University

背景&问题动机

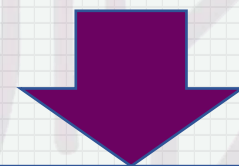
允公允能 日新月异

研究背景

- 容器是云计算与 Serverless 的核心基础设施
- 主流容器实现高度依赖 Linux 内核机制
- Linux 并非为强隔离、多租户场景设计

问题动机

- 容器隔离基于“事后限制”
- 内核复杂度与攻击面持续扩大
- 容器逃逸漏洞频繁出现



如果操作系统从设计上就遵循最小权限原则，是否还需要复杂而脆弱的容器隔离机制？



南開大學
Nankai University

Linux容器的本质问题

2

允公允能 日新月异

Linux 容器的本质问题

- 隔离依赖事后限制，而非原生设计
- Ambient authority 使权限边界难以精确控制
- namespace / seccomp 本身扩大内核攻击面
- TCB 过大，安全性难以验证
- 强隔离语义与单体内核架构存在结构性冲突

Linux 容器试图在一个权限过大、功能过于复杂的内核之上，通过附加限制来实现强隔离，这种模式在结构上天然脆弱。



南開大學

Nankai University

为什么是微内核？

3

允公允能 日新月异

为什么是微内核?

Linux

VS

微内核



南开大学
Nankai University

➤ 传统 Linux 内核的设计前提

面向单系统、单管理员环境

进程默认拥有广泛系统权限

➤ 容器隔离的实现方式

通过 namespace 虚拟系统视图

通过 seccomp / cgroup 事后限制行为

➤ 根本矛盾

隔离依赖复杂的内核限制逻辑

安全机制本身扩大内核攻击面

可信计算基 (TCB) 规模过大

➤ 微内核的核心设计

内核仅保留最小必要功能

操作系统服务运行在用户态

➤ Capability 权限模型

资源访问必须显式授权

不存在全局命名空间或隐式权限

未授权即不可见、不可访问

➤ 安全与隔离优势

隔离由设计保证，而非事后限制

显著缩小可信计算基 (TCB)

隔离机制不再成为攻击面



南開大學
Nankai University

MettEagle的整体架构

4

允公允能 日新月异

MettEagle 整体架构概览



南开大学
Nankai University

整体分层图

云应用与 Serverless 应用层
(函数 / 服务)

MettEagle 运行时层

➤ Phlox (FaaS 运行时)

➤ 隔离单元服务 (Compartment Service)

用户态操作系统服务层
文件系统 | 网络 | 程序加载器

L4Re 微内核
调度 | 地址空间 | 进程间通信

➤ 系统运行基础

MettEagle 构建在 L4Re 微内核操作系统之上
微内核仅负责：线程调度、地址空间管理、进程间通信

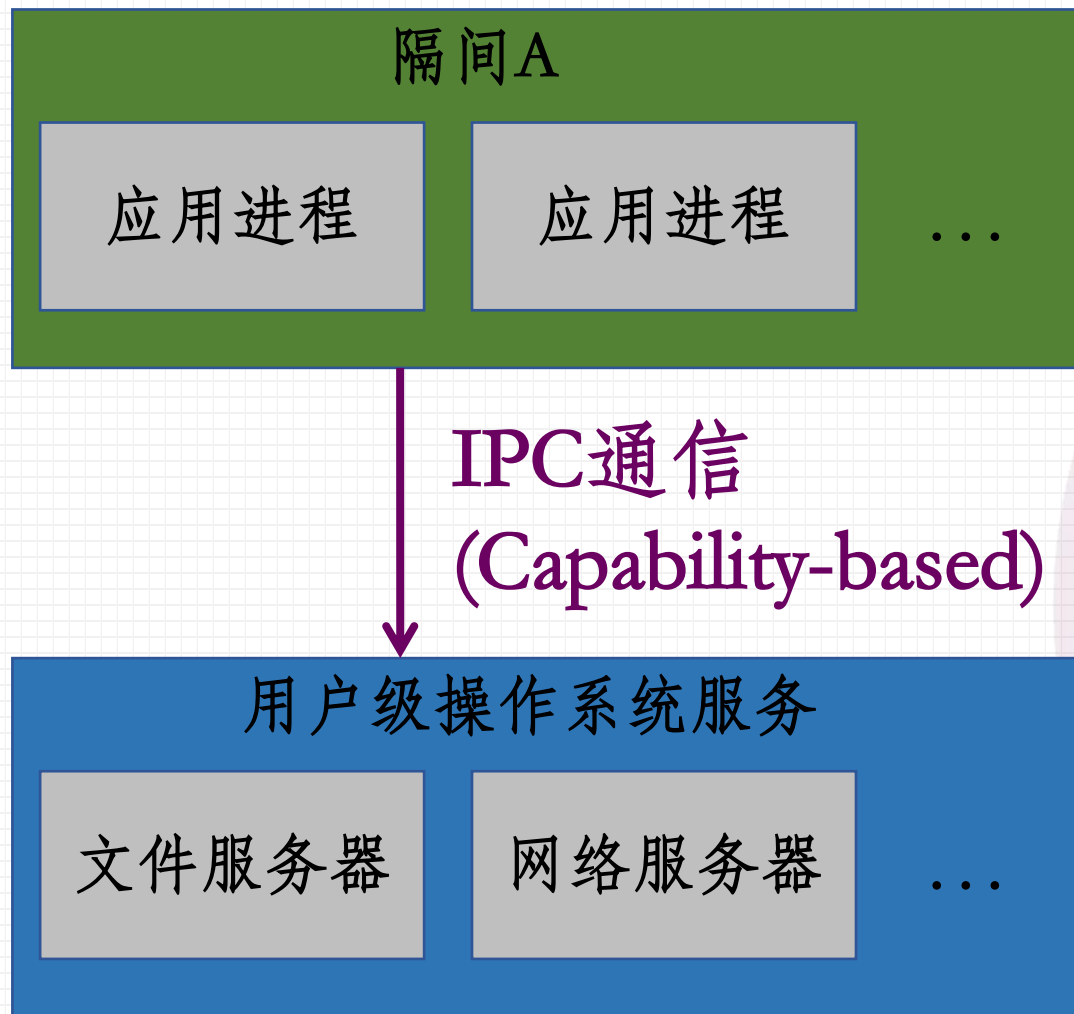
➤ 架构分层

- 用户应用运行在 Compartment (隔离单元) 中
- 操作系统服务 (文件、网络等) 运行在用户态
- MettEagle 作为容器运行时，负责创建与管理隔离环境

MettEagle 核心组件与隔离机制



南开大学
Nankai University



➤ Compartment (隔离单元)

MettEagle 中的基本隔离实体

包含：一组任务、独立的地址空间、显式分配的 capabilities 架构分层

➤ 核心组件

Compartment Service

- 创建与销毁 Compartment
- 分配 capability (文件、网络、内存等)
- 管理生命周期与资源回收

Phlox (高层运行时)

- 面向 Serverless / FaaS 场景
- 负责函数加载、调度与远程调用
- 类似于 containerd + FaaS runtime 的结合



南開大學
Nankai University

隔离机制对比

5

允公允能 日新月异

linux

- 命名空间
隔离PID、挂载、网络、用户等
- 控制组
cgroups 控制资源消耗
- seccomp-bpf
过滤系统调用
- ✗ root 权限仍可能带来安全风险

L4Re/MettEagle

- 基于能力（Capability）的访问控制
- 核心功能在用户态组件中运行
- 完全的进程隔离
每个容器拥有独立能力，遵循最小权限原则
- ✓ 没有全能 root，防止权限提升

VS



南開大學
Nankai University

安全性評估

6

允公允能 日新月異

安全性评估(1/2): 可信计算基



南开大学
Nankai University

没有精确的安全度量 → 使用代理指标

Component of MettEagle	SLOC
L4Re kernel	41,406
Sigma0 (root pager)	1,249
Moe (root task)	4,810
Ned (system loader)	2,870
IO (PCI driver)	22,012
LSMM (parallelized memory manager)	4,289
PROMFS (parallelized boot FS)	780
SPAFS (file system)	501
LUNA (NIC driver & UDP stack)	8,735
Compartment Service	1,793
Phlox (FaaS launcher)	826
Σ	89,271

Table 1: TCB size of MettEagle container infrastructure.

Component of Linux	SLOC
Linux kernel v6.7.4 (including ramfs)	1,454,290
NIC driver module	31,675
containerd	922,901
runc	290,946
Σ	2,699,812

Table 2: TCB size of Linux container infrastructure.

更小的 TCB → 更少的漏洞 & 更易审计

CVE 研究方法

33 个高危 Linux 容器漏洞 (CVE)
(涉及 seccomp / 命名空间 / 控制组)

举例 seccomp-bpf

CVE 分类结果

-CVE 对 MettEagle 的影响:

完全缓解 (FM) : 多数

部分缓解 (PM) : 部分

未缓解 (N) : 少数

“为什么会这样” ?

根本原因: Linux 依赖复杂的内核机制 MettEagle 通过能力 (capabilities) 来强制实现隔离

Linux 的 seccomp-bpf 基于 eBPF, 而 eBPF 本身就是一个运行在内核里的解释器, 代码量超过 5 万行; MettEagle 不需要这种过滤机制, 因为没有 capability 的接口根本不可见。



南開大學
Nankai University

性能評估

7

允公允能 日新月異

评估目标:

- 评估 L4Re 隔离舱 (MettEagle) 的隔离开销
- 关注操作系统层面的机制，不使用 warm start 等启动优化

核心问题:

- 容器启动延迟 (Startup Latency)
- I/O 性能 (网络)
- 应用端到端性能 (SeBS)

对比系统

• Linux 进程
性能基线

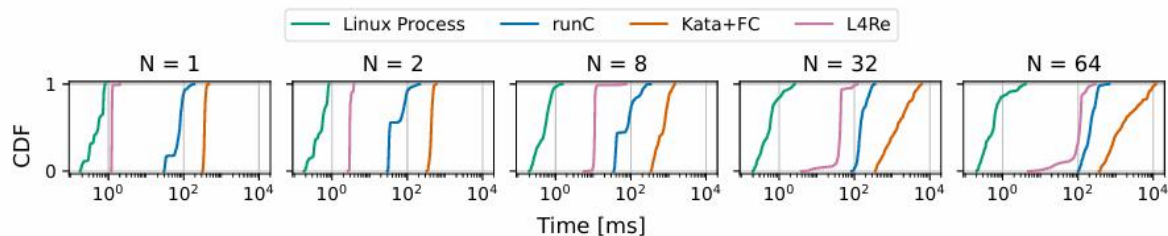
• runC
Linux 容器

• Kata + Firecracker
轻量虚拟机

• MettEagle
L4Re 隔离舱

冷启动延迟 (Cold Start)

- Linux 进程: $\approx 200 \mu\text{s}$ (最优情况)
- runC: $\approx 70 \text{ ms}$ (单容器)
- **MettEagle**: $\approx 1 \text{ ms}$ (N=1)
- 高并发启动 (64 个容器):
 - MettEagle: $\approx 100 \text{ ms}$
 - runC: $\approx 200 \text{ ms}$



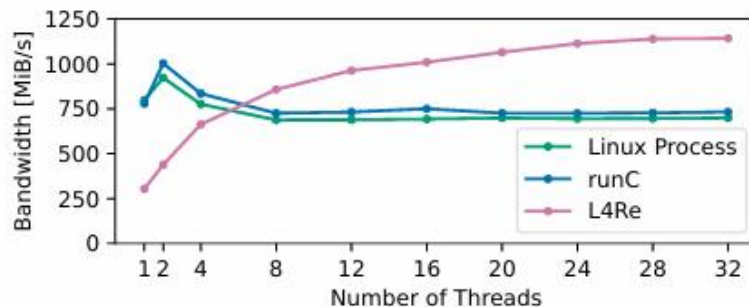
在高并发容器启动场景下, MettEagle 比 runC 具有更好的可扩展性

允公允能 日新月异

I/O 性能

网络 I/O (UDP)

- 延迟: 所有平台 $\approx 40 \mu\text{s}$
- 带宽:
 - 低并发下略低 (单核用户态 NIC)
 - 高并发可跑满 10Gbps



用户态 I/O 并不是根本性的性能瓶颈

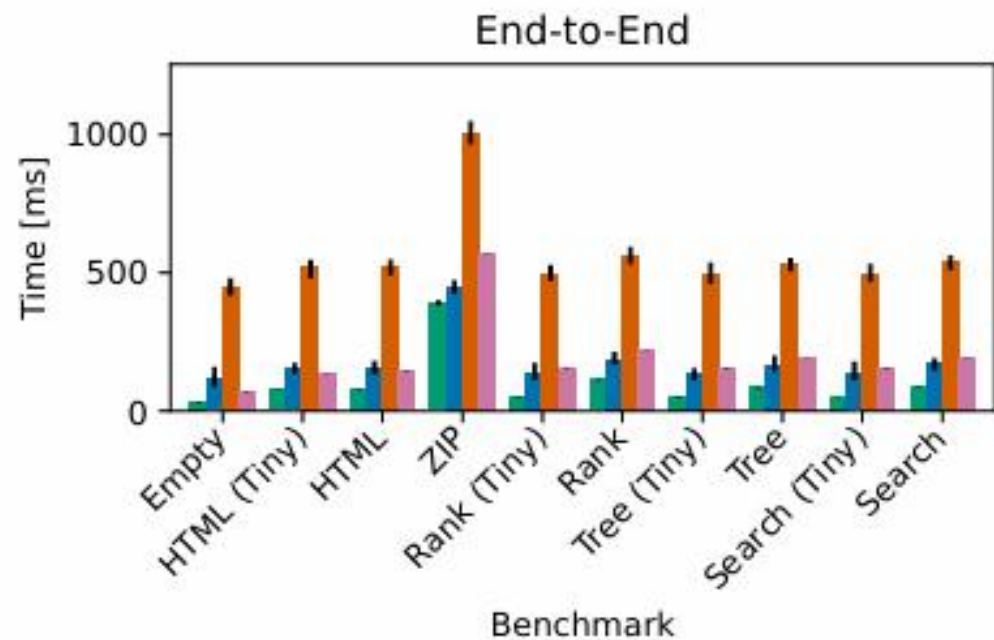
应用性能评估 (SeBS, Python)

•端到端延迟:

- 大多数 benchmark 与 runC 差距 $\leq 15\%$
- 某些场景甚至更快 (如 HTML)
- 始终优于 Kata

•性能差距的原因:

- L4Re 文件系统操作较慢
- Python 模块加载开销大
- 不是隔离机制本身导致的



微内核架构是可行的, 瓶颈来自实现层面



南开大学
Nankai University

总结与收获

8

允公允能 日新月异

核心结论

- 微内核架构 (L4Re) 可以实现安全、高隔离的容器环境。
- 与 Linux 容器相比, MettEagle TCB 小, 内核特权减少, 安全性更强。

性能亮点

- 容器启动速度可与 Linux 容器竞争 (甚至在一些场景更快)
- 微内核设计的简洁性允许进一步优化 I/O 和 Python 运行时性能。

安全性亮点

- CVE 调研显示大部分 Linux 高危漏洞在微内核系统中被完全或部分缓解。
- 基于能力 (capability) 的访问控制减少权限滥用风险。

设计启示 / 应用前景

- 微内核容器适用于高安全、云原生、多租户环境。
- 安全与性能的平衡是可行的, 不必牺牲安全来换性能。





汇报结束 感谢聆听！

THANKS FOR YOUR LISTENING!

小组成员：于昊汐 陈安琪 孙俪菲

日期：2025-12-16